Assignment-1: Benchkit

Milan Lagae

University Name: Vrije Universiteit Brussel

Faculty: Sciences and Bioengineering Sciences

Course: Performance Analysis & Evaluation

May 10, 2026

Contents

1. Intro

The benchmark chosen for porting to the updated Benchkit V2 campaign API is the `cuda_samples_matmul`. To start, context of the chosen benchmark will be discussed in Section 2. Modifications made will be discussed in Section 3, and use of a the `ncu` command wrapper in Section 4. Discussion of benchmark results will be done in Section 5.

2. Context

The chosen benchmark to port to the updated API is `cuda_matmul`. The benchmark measures the performance of matrix multiplication on Nvidia GPU's. Matrix multiplication are regularly used to compare the performance of different (Nvidia) GPU's.

The benchmark takes the dimensions of the 2 matrixes that have to be multiplied as input parameters.

3. Bench

This section will briefly discuss the changes made to the benchmark implementation.

3.1. Fetch

The fetch stage of the benchmark has been updated to clone the repository containing the benchmark code. The repository is cloned into it's own directory.

3.2. Build

The build stage of the benchmark is currently setup to only build the `matrixMul.cu` kernel. The `cmake` & `make` commands are sequentially executed. First the `build` directory is created in the folder: `/Samples/0_Introduction/matrixMul/` .

Inside of the `/Samples/0_Introduction/matrixMul/build` folder, the `cmake` command is executed. Lastly, the `make` command is executed in the `/Samples/0_Introduction/matrixMul/build` folder

3.3. Run

This stage executes a single `matrixMul` Cuda kernel. Existing code has been reused and updated. The command in Listing 1 is executed, note that the `ma_width` is reused twice, this to ensure that both matrices can be multiplied with each other.

```
1 run_command = [
2     "./matrixMul",
3     f"-wA={ma_width}",
4     f"-hA={ma_height}",
5     f"-wB={mb_width}",
6     f"-hB={ma_width}",
7 ]
```

Listing 1: Run - Command

3.4. Collect

The existing collect has been slightly modified. The original collect phase code has been reused. The updated modification, is the addition of 3 new metric fields, as follows:

- dim_a: Matrix A dimensions.
- dim_b: Matrix B dimensions.
- dim_a_x_dim_b: Matrix A & B dimensions combined.

4. Command wrapper

The chosen wrapper for the updated benchmark is the `ncu2.py` wrapper, which is the cli version of NVIDIA Nsight Compute[1].

By using the GPU's hardware performance counters, more information from the execute kernel can be gained.

[1] https://developer.nvidia.com/nsight-compute

5. Analysis

This section will discuss analyzing the results of executing the updated benchmark for the Cuda matrix multiplication example.

5.1. Platform

Due to platform availability, the benchmarks were executed on a Ubuntu 22.04 WSL image on Windows 11. More information can be found in Table 1.

Part	Value
CPU	Ryzen 9 5950X
GPU	RTX 3070
Driver Version	595.97
RAM	64GB (3200 Mhz)
OS	Windows 11 - Version 10.0.22631 Build 22631
WSL Version	2
WSL Distro	24.04

Table 1: Desktop Specifications

5.2. Base

Included in the output of the base cuda kernel performance metrics are the following values:

- ma_width: Matrix A size.
- ma_height: Matrix A size.
- mb_width: Matrix B size.
- ma_width: Matrix A size.
- dim_a: Matrix A dimensions.
- dim_b: Matrix B dimensions.
- dim_a_x_dim_b: Matrix A & B dimensions combined.
- GFlops/s: Compute throughput per second.
- Time ms: Execution time.
- Ops: Total number of operations executed.
- Workgroup Size: Indicates the number of threads in a work group.

5.2.1. Graphs

The run variables used in the benchmark can be found in Listing 2.

```
1 "ma_width": [32, 64, 128],
2 "ma_height": [32, 64, 128],
3 "mb_width": [32, 64, 128],
```

Listing 2: Base - Run Variables

The matrix A size vs execution time is charted in the figures Figure 1 and Figure 1.

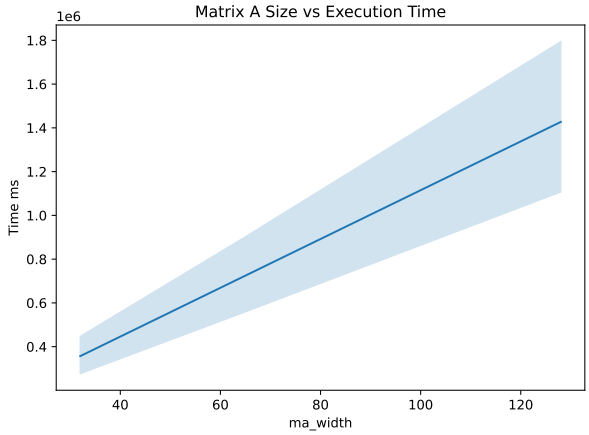

Figure 1: Old - Size vs Execution Time

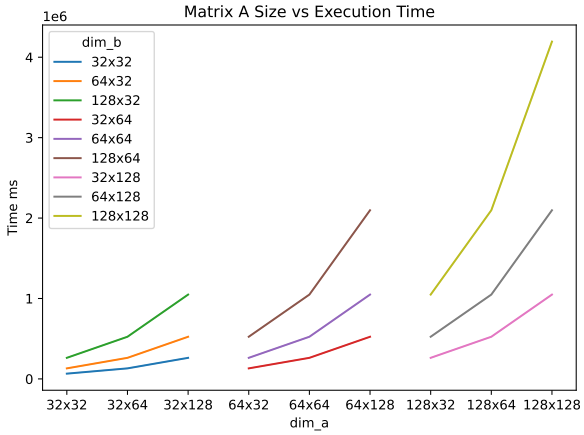

Figure 2: New - Size vs Execution Time

While Figure 2 gives a general idea of how the execution time rises with Matrix A Width, the information of the dimension of both matrixes are lost. For this reason the additional metric fields in the collect phase have been added. The updated chart with these additional fields are shown in Figure 1.

Furthermore, the computational throughput of the matrix multiplication kernel can be charted as seen in Figure 3 and Figure 4.

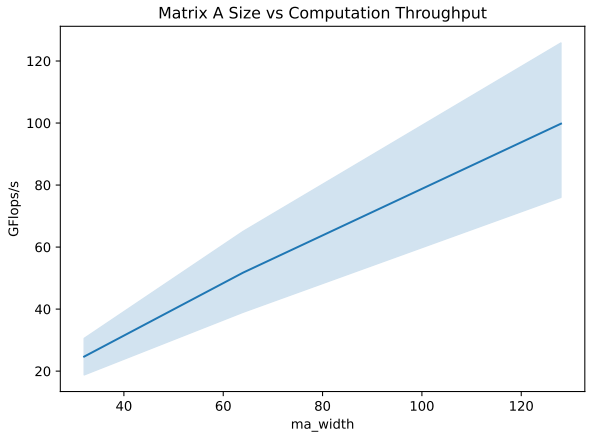

Figure 3: Old - Size vs Throughput

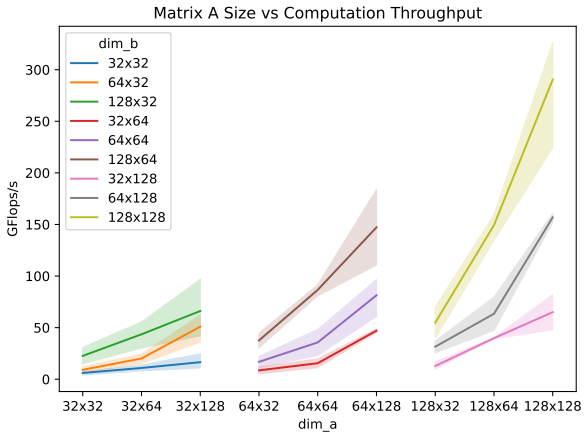

Figure 4: New - Size vs Throughput

Using the existing information gathered during the collect phase, the throughput vs the matrix A width is graphed in Figure 3, but again some information is lost. For this reason, using the additional metric fields added in the collect phase, the information is illustrated in Figure 4.

5.3. Wrapper

The default metric included in the wrapper for analysis is: `smsp__sass_l1tex_tags_mem_global`. Using the command `ncu --query-metrics --metrics smsp__sass_l1tex_tags_mem_global` the description of this metric can be queried. The description reads as follows: "# of L1 cache tag lookups generated by global memory instructions".

5.3.1. Graphs

The run variables used in the benchmark can be found in Listing 3.

```
1 "ma_width": [32, 64, 128],
2 "ma_height": [32, 64, 128],
3 "mb_width": [32, 64, 128],
```

Listing 3: Wrapper - Run Variables

The measured NCU metric is illustrated in the graphs Figure 5 and Figure 6.

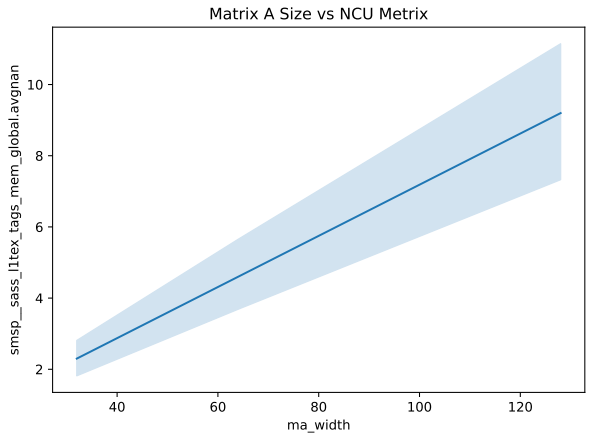

Figure 5: Old - Size vs NCU Metric

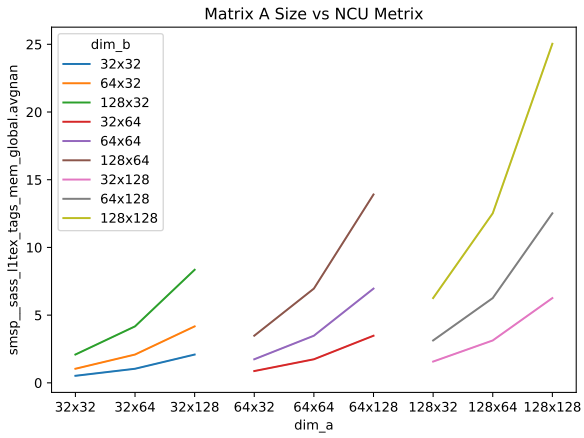

Figure 6: New - Size vs NCU Metric

The additional information gathered in the collect phase, again illustrates it usefulness in Figure 6. Plotting using the new information has it limits. When increasing the size & number of run variables, the information becomes too much to be properly displayed on the x-axis & inside of the legend of the chart.

5.4. Pretty

Using the `pretty` value on the campaign the graphs can be improved again. The figures can be seen in Figure 7 & Figure 8.

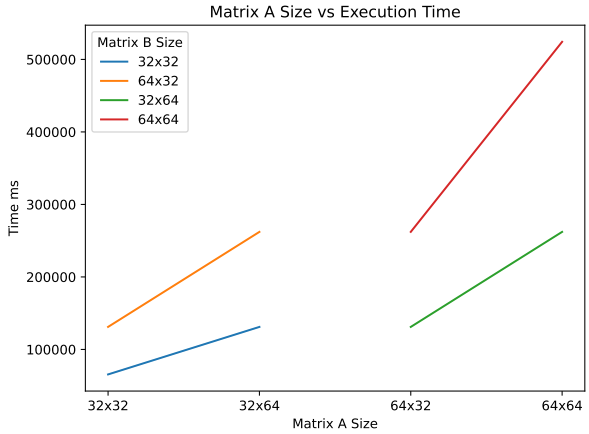

Figure 7: Pretty - Matrix A Size vs Time

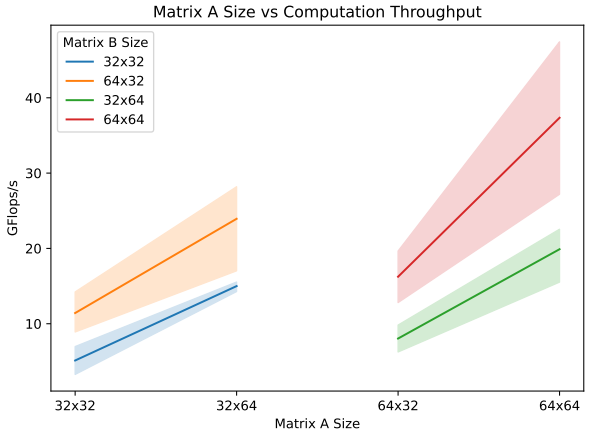

Figure 8: Pretty - Matrix A Size vs Throughput

And for the `ncu` wrapper with measured metric in Figure 9 & Figure 10.

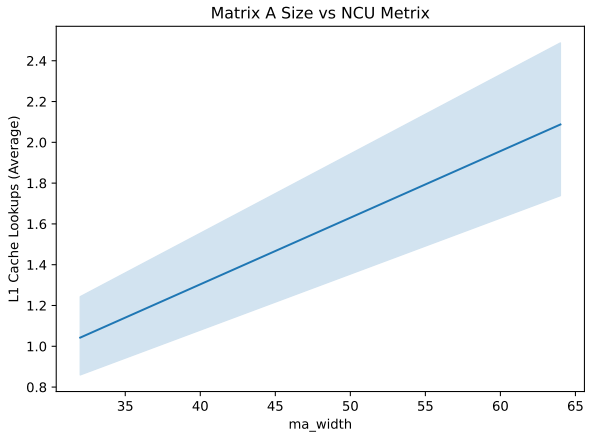

Figure 9: Pretty - Matrix A Size vs NCU Metric

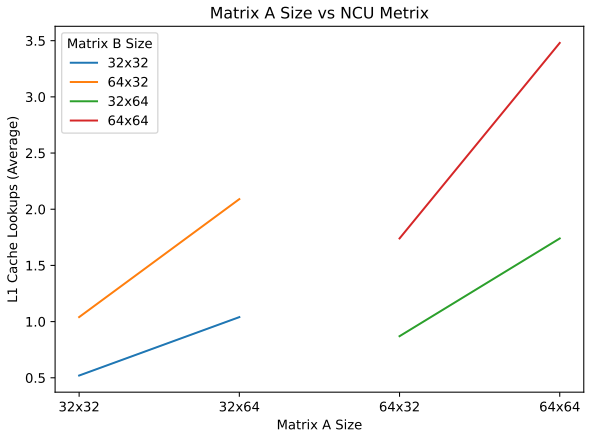

Figure 10: Pretty - Matrix A Size vs NCU Metric